

LINCS: A Linear Constraint Solver for Molecular Simulations

BERK HESS,¹ HENK BEKKER,² HERMAN J. C. BERENDSEN,¹
JOHANNES G. E. M. FRAAIJE¹

¹Bioson Research Institute, Lab of Biophysical Chemistry, University of Groningen, Nijenborgh 4, 9747 AG Groningen, The Netherlands

²Department of Computer Science, University of Groningen, Groningen, The Netherlands

Received 12 January 1996; accepted 15 February 1997

ABSTRACT: In this article, we present a new LINear Constraint Solver (LINCS) for molecular simulations with bond constraints. The algorithm is inherently stable, as the constraints themselves are reset instead of derivatives of the constraints, thereby eliminating drift. Although the derivation of the algorithm is presented in terms of matrices, no matrix matrix multiplications are needed and only the nonzero matrix elements have to be stored, making the method useful for very large molecules. At the same accuracy, the LINCS algorithm is three to four times faster than the SHAKE algorithm. Parallelization of the algorithm is straightforward. © 1997 John Wiley & Sons, Inc. *J Comput Chem* 18: 1463–1472, 1997

Keywords: constraints; molecular dynamics; Langevin dynamics; SHAKE

Introduction

In classical molecular simulation methods, such as molecular dynamics (MD) or Langevin dynamics (LD), the time step is limited by bond oscillations. These oscillations have a relatively high frequency and low amplitude. By replacing the bond vibrations with holonomic constraints the time step in molecular simulations can be increased by a factor of four. Constraints are often considered a more faithful representation of the

physical behavior of bond vibrations, which are almost exclusively in their vibrational ground state.

Because bonds in molecules are coupled, resetting coupled constraints after an unconstrained update is a nonlinear problem. Many algorithms have been proposed for solving this problem. The most widely used algorithm for large molecules is SHAKE.¹ For small molecules, SETTLE is a faster algorithm.² Both these methods reset bonds (and angles) to prescribed values by moving the bonded particles parallel to the old bond directions. SETTLE solves this problem analytically, but for larger molecules this is too complicated. SHAKE is an iterative method. Sequentially all the bonds are set

Correspondence to: J. G. E. M. Fraaije

to the correct length. Because the bonds are coupled, this procedure has to be repeated until the desired accuracy is reached. The number of iterations can be decreased using overrelaxation.³ SHAKE is simple and numerically stable because it resets all constraints within a prescribed tolerance. It has been shown that, when the iteration is carried to convergence, SHAKE in combination with Verlet is symplectic and time reversible.⁴ SHAKE has the drawback that no solutions may be found when displacements are large: because the coupled bonds are handled one by one, correcting one bond may tilt a coupled bond so far that the method does not converge. Due to its iterative nature it is difficult to parallelize SHAKE.⁵ Therefore, there is need for a fast, reliable constraint algorithm in molecular simulations.

The constraint problem reduces to a linear matrix equation if the second derivatives of the constraint equations are set to zero. However, in a finite discretization scheme corrections are necessary to achieve accuracy and stability. Several algorithms have been proposed, including the EEM method of Edberg et al.,⁶ who applied a penalty function; a modification of EEM by Baranyai and Evans,⁷ applying damping corrections, and the "noniterative SHAKE" method of Yoneya et al.,^{8,9} approximating the constraint equations rather than their second derivatives.

In this article we present a new LINear Constraint Solver (LINCS) for molecular simulations with bond constraints. LINCS is similar to EEM with a few practical differences. We have implemented an efficient solver for the matrix equation, a velocity correction that prevents rotational lengthening and a length correction that improves accuracy and stability. The LINCS algorithm is worked out in detail for application to a leap-frog or Verlet-type algorithm for molecular dynamics. The application to position Langevin dynamics is briefly discussed. The sparse constraint coupling matrix is inverted using a power series, which converges rapidly because the constraining influence is relatively local; therefore, parallelization of the algorithm is straightforward. The resulting method is three to four times faster than SHAKE at the same accuracy.

Projection Method

We consider a system of N particles, with positions given by a $3N$ vector $\mathbf{r}(t)$. The equations of

motion are given by Newton's law:

$$\frac{d^2\mathbf{r}}{dt^2} = \mathbf{M}^{-1}\mathbf{f} \quad (1)$$

where $\mathbf{f}$ is the $3N$ force vector and $\mathbf{M}$ is a $3N \times 3N$ diagonal matrix, containing the masses of the particles. In general, a system is constrained by K time-independent constraint equations:

$$g_i(\mathbf{r}) = 0 \quad i = 1, \dots, K \quad (2)$$

As shown by Bekker¹⁰ the constrained system can still be described by $3N$ second-order differential equations in Cartesian coordinates. The proof runs as follows.

The system is constrained according to the principle of least action.¹¹ In this approach, the constraints are added as a zero term to the potential $V(\mathbf{r})$, multiplied by Lagrange multipliers $\lambda_i(t)$:

$$-\mathbf{M} \frac{d^2\mathbf{r}}{dt^2} = \frac{\partial}{\partial \mathbf{r}} (V - \boldsymbol{\lambda} \cdot \mathbf{g}) \quad (3)$$

A new notation is introduced for the gradient matrix of the constraint equations, which appears on the right-hand side of:

$$B_{hi} = \frac{\partial g_h}{\partial r_i} \quad (4)$$

Notice that $\mathbf{B}$ is a $K \times 3N$ matrix, it contains the directions of the constraints. Eq. (3) can now be simplified to give:

$$-\mathbf{M} \frac{d^2\mathbf{r}}{dt^2} + \mathbf{B}^T \boldsymbol{\lambda} + \mathbf{f} = 0 \quad (5)$$

Because the constraint equations are zero, the first and second derivatives of the constraints are also zero:

$$\frac{d\mathbf{g}}{dt} = \mathbf{B} \frac{d\mathbf{r}}{dt} = 0 \quad (6)$$

$$\frac{d^2\mathbf{g}}{dt^2} = \mathbf{B} \frac{d^2\mathbf{r}}{dt^2} + \frac{d\mathbf{B}}{dt} \frac{d\mathbf{r}}{dt} = 0 \quad (7)$$

To solve for $\boldsymbol{\lambda}$, left-multiply eq. (5) with $\mathbf{B}\mathbf{M}^{-1}$, and use eq. (7) to get:

$$\frac{d\mathbf{B}}{dt} \frac{d\mathbf{r}}{dt} + \mathbf{B}\mathbf{M}^{-1}\mathbf{B}^T \boldsymbol{\lambda} + \mathbf{B}\mathbf{M}^{-1}\mathbf{f} = 0 \quad (8)$$

Thus, the constraint forces are:

$$\mathbf{B}^T \boldsymbol{\lambda} = -\mathbf{B}^T (\mathbf{B} \mathbf{M}^{-1} \mathbf{B}^T)^{-1} \mathbf{B} \mathbf{M}^{-1} \mathbf{f} - \mathbf{B}^T (\mathbf{B} \mathbf{M}^{-1} \mathbf{B}^T)^{-1} \frac{d\mathbf{B}}{dt} \frac{d\mathbf{r}}{dt} \quad (9)$$

Substituting this into eq. (5) gives the constrained equations of motion. Using the abbreviation $\mathbf{T} = \mathbf{M}^{-1} \mathbf{B}^T (\mathbf{B} \mathbf{M}^{-1} \mathbf{B}^T)^{-1}$ these are:

$$\frac{d^2 \mathbf{r}}{dt^2} = (\mathbf{I} - \mathbf{T} \mathbf{B}) \mathbf{M}^{-1} \mathbf{f} - \mathbf{T} \frac{d\mathbf{B}}{dt} \frac{d\mathbf{r}}{dt} \quad (10)$$

$\mathbf{I} - \mathbf{T} \mathbf{B}$ is a projection matrix that sets the constrained coordinates to zero, $\mathbf{B} \mathbf{M}^{-1} \mathbf{f}$ is a K vector of second derivatives of the bond lengths in the direction of the bonds, $\mathbf{T}$ is a $3N \times K$ matrix that transforms motions in the constrained coordinates into motions in Cartesian coordinates, without changing the equations of motion of the unconstrained coordinates. The last term in eq. (10) represents centripetal forces caused by rotating bonds. If the constraints are satisfied in the starting configuration, eq. (10) will conserve the constraints.

Constrained Leap-Frog Algorithm

A straightforward discretization in a leap-frog method uses a half-timestep difference between the discretization of the first and last derivatives in eq. (10):

$$\frac{\mathbf{v}_{n+\frac{1}{2}} - \mathbf{v}_{n-\frac{1}{2}}}{\Delta t} = (\mathbf{I} - \mathbf{T}_n \mathbf{B}_n) \mathbf{M}^{-1} \mathbf{f}_n - \mathbf{T}_n \frac{\mathbf{B}_n - \mathbf{B}_{n-1}}{\Delta t} \mathbf{v}_{n-\frac{1}{2}} \quad (11)$$

$$\frac{\mathbf{r}_{n+1} - \mathbf{r}_n}{\Delta t} = \mathbf{v}_{n+\frac{1}{2}} \quad (12)$$

The discretization of the last term in eq. (11) at $t_{n-\frac{1}{2}}$ is the actual linearization of the problem. Because the right-hand side of eq. (11) does not depend on $t_{n+\frac{1}{2}}$, the new velocities can be calculated directly. However, the half-timestep difference removes the correction for centripetal forces. This correction has to be done afterwards. If, in eq. (11), the term $\mathbf{B}_{n-1} \mathbf{v}_{n-\frac{1}{2}}$ is zero, the equation simplifies to:

$$\mathbf{v}_{n+\frac{1}{2}} = (\mathbf{I} - \mathbf{T}_n \mathbf{B}_n) (\mathbf{v}_{n-\frac{1}{2}} + \Delta t \mathbf{M}^{-1} \mathbf{f}_n) \quad (13)$$

and because $\mathbf{I} - \mathbf{T}_n \mathbf{B}_n$ is the projection matrix that sets the constrained coordinates to zero, $\mathbf{B}_n \mathbf{v}_{n+\frac{1}{2}} = 0$. This proves that the velocities in the bond direc-

tions stay at zero. Thus, if we set $\mathbf{B}_0 \mathbf{v}_{\frac{1}{2}} = 0$, eq. (13) can be used instead of eq. (11).

The derived algorithm is correct, but not stable. Numerical errors can accumulate, which leads to drift, because the second derivatives of the constraints were set to zero instead of the constraints themselves. This problem can be overcome by making a small change to the expression for the constraint forces. For holonomic constraints the constraint equations can be chosen as:

$$g_i(\mathbf{r}) = |\mathbf{r}^{i_1} - \mathbf{r}^{i_2}| - d_i = 0 \quad i = 1, \dots, K \quad (14)$$

where d_i is the length of the bond between atoms i_1 and i_2 , and the vectors with superscripts are the positions of atoms. To get a stable and efficient algorithm a term is added to eq. (13):

$$\mathbf{v}_{n+\frac{1}{2}} = (\mathbf{I} - \mathbf{T}_n \mathbf{B}_n) (\mathbf{v}_{n-\frac{1}{2}} + \Delta t \mathbf{M}^{-1} \mathbf{f}_n) - \frac{1}{\Delta t} \mathbf{T}_n (\mathbf{B}_n \mathbf{r}_n - \mathbf{d}) \quad (15)$$

The added term should physically be zero, as it is the real distance between bounded atoms minus the prescribed bond lengths. Of course, in a simulation, this term is almost never exactly zero, so adding this term eliminates accumulation of numerical errors and makes the method stable. Substituting eq. (15) into eq. (12) gives the constrained new positions:

$$\mathbf{r}_{n+1} = (\mathbf{I} - \mathbf{T}_n \mathbf{B}_n) (\mathbf{r}_n + \Delta t \mathbf{v}_{n-\frac{1}{2}} + (\Delta t)^2 \mathbf{M}^{-1} \mathbf{f}_n) + \mathbf{T}_n \mathbf{d} \quad (16)$$

This is actually how the method is implemented. It calculates the new positions as in the unconstrained case. Then it sets the components of the distances in the direction stored in $\mathbf{B}_n$ exactly to the prescribed lengths. The new velocity is calculated from the positions with eq. (12). The corresponding Verlet algorithm is derived by substituting $\mathbf{v}_{n-\frac{1}{2}}$ from eq. (12) into eq. (16).

The main drawback of the method presented so far is that it does not set the real bond lengths to the prescribed lengths, but the projection of the new bonds onto the old directions of the bonds. Thus, the bond lengths are increased by a factor $1/\cos \theta$, where θ is the angle over which a bond rotates in one timestep. In an "average" MD simulation of a protein or peptide with a 2-fs timestep the relative deviation is 0.0005 (rms). Notice that the error over n steps is still 0.0005, because accumulation of errors is prevented by adding the

correction term. To get a higher accuracy another projection can be applied, again using the old bond directions.

To correct for the rotation of bond i , the projection of the bond on the old direction is set to:

$$p_i = \sqrt{2d_i^2 - l_i^2} \quad (17)$$

where l_i is the bond length after the first projection. The corrected positions are:

$$\mathbf{r}_{n+1}^* = (\mathbf{I} - \mathbf{T}_n \mathbf{B}_n) \mathbf{r}_{n+1} + \mathbf{T}_n \mathbf{p} \quad (18)$$

Figure 1 shows a schematic picture of how the algorithm works for one bond. For proteins with a 2-fs timestep the relative accuracy is 0.000001 (rms). The error is not zero, because bonds are rotated slightly due to the coupling. So, actually, this correction procedure should be iterative, but the accuracy after one iteration is high enough for all purposes. The constraints cannot be reset by moving atoms in the old bond directions in the extreme case that for a certain bond l_i^2 is larger than $2d_i^2$. However, setting p_i to zero results in a "solution" with bondlengths that are as close as possible to the prescribed lengths.

Just like SHAKE, LINCS calculates the new constrained positions from the old positions and the new unconstrained positions. Thus, the same methods can be used to calculate the virial and free energy differences. However, the free energy

calculation for the constraints can be implemented simply and efficiently in the LINCS algorithm. The formulas are given in the Appendix.

Implementation

Eq. (16) is of the shape:

$$\begin{aligned} \mathbf{r}_{n+1} &= (\mathbf{I} - \mathbf{T}_n \mathbf{B}_n) \mathbf{r}_{n+1}^{unc} + \mathbf{T}_n \mathbf{d} \\ &= \mathbf{r}_{n+1}^{unc} - \mathbf{M}^{-1} \mathbf{B}_n (\mathbf{B}_n \mathbf{M}^{-1} \mathbf{B}_n^T)^{-1} (\mathbf{B}_n \mathbf{r}_{n+1}^{unc} - \mathbf{d}) \end{aligned} \quad (19)$$

where $\mathbf{r}_{n+1}^{unc}$ are the new positions after an unconstrained update. Half of the CPU time goes to inverting the constraint matrix $\mathbf{B}_n \mathbf{M}^{-1} \mathbf{B}_n^T$, which has to be done every timestep. This $K \times K$ matrix has $1/m_{i_1} + 1/m_{i_2}$ on the diagonal. The off-diagonal elements are only nonzero when two bonds are connected, then the element is $\cos \phi / m_c$, where m_c is the mass of the atom connecting the two bonds and ϕ is the angle between the bonds. To make the inversion easier, a $K \times K$ matrix $\mathbf{S}$ is introduced, which is the inverse square root of the diagonal of $\mathbf{B}_n \mathbf{M}^{-1} \mathbf{B}_n^T$:

$$\mathbf{S} = \text{Diag} \left(\sqrt{\frac{1}{m_{i_1}} + \frac{1}{m_{i_2}}}, \dots, \sqrt{\frac{1}{m_{i_{K_1}}} + \frac{1}{m_{i_{K_2}}}} \right) \quad (20)$$

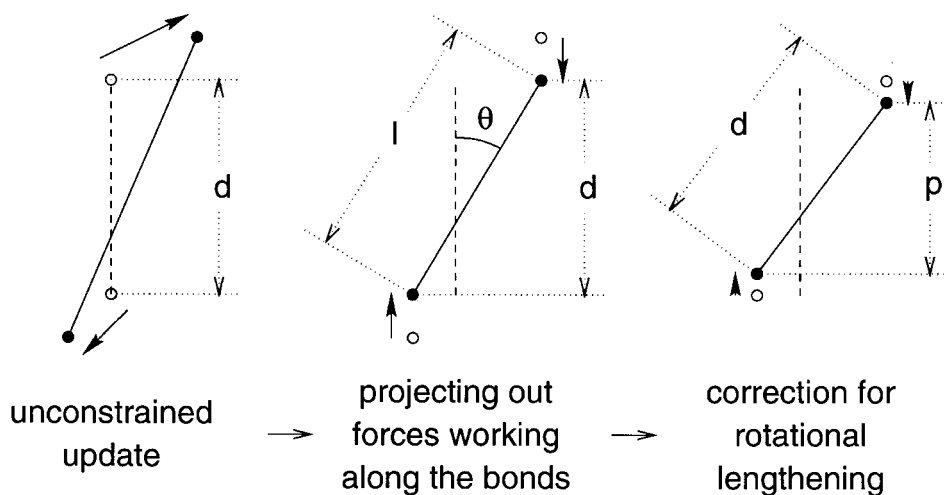


FIGURE 1. Schematic picture showing the three position updates needed for one timestep. The dashed line is the old bond of length d , the solid lines are the new bonds. $l = d \cos \theta$ and $p = (2d^2 - l^2)^{1/2}$.

This matrix is used to convert the diagonal elements of the coupling matrix to one

$$\begin{aligned}(\mathbf{B}_n \mathbf{M}^{-1} \mathbf{B}_n^T)^{-1} &= \mathbf{S} \mathbf{S}^{-1} (\mathbf{B}_n \mathbf{M}^{-1} \mathbf{B}_n^T)^{-1} \mathbf{S}^{-1} \mathbf{S} \\&= \mathbf{S} (\mathbf{S} \mathbf{B}_n \mathbf{M}^{-1} \mathbf{B}_n^T \mathbf{S})^{-1} \mathbf{S} \\&= \mathbf{S} (\mathbf{I} - \mathbf{A}_n)^{-1} \mathbf{S}\end{aligned}\quad (21)$$

The matrix $\mathbf{A}_n$ is symmetric and sparse and has zeros on the diagonal. Thus, a simple trick can be used to calculate the inverse:

$$(\mathbf{I} - \mathbf{A}_n)^{-1} = \mathbf{I} + \mathbf{A}_n + \mathbf{A}_n^2 + \mathbf{A}_n^3 + \cdots \quad (22)$$

This inversion method is only valid if the absolute values of all the eigenvalues of $\mathbf{A}_n$ are smaller than one. In molecules with only bond constraints the connectivity is so low that this will always be true, even if ring structures are present. Problems can arise in angle-constrained molecules. By constraining angles with additional distance constraints multiple small ring structures are introduced. This gives a high connectivity, leading to large eigenvalues. For example, the largest eigenvalue for angle-constrained butane is 0.8, but for angle-constrained pentane this is 1.2. However, the projection method itself is still applicable, if the constraint coupling matrix is inverted with another method.

The inversion through an expansion is efficient, because the inverse itself is not needed, only the product of the inverse with a vector. This product can be calculated without multiplying matrices. $\mathbf{A}_n^{i+1} \mathbf{a}$ can be written as $\mathbf{A}_n (\mathbf{A}_n^i \mathbf{a})$, showing that this product can be calculated by multiplying vectors with $\mathbf{A}_n$.

The error made by truncating the series expansion in eq. (22) is quite clear. The first power of $\mathbf{A}_n$ gives the coupling effects of neighboring bonds. The second power gives the coupling effects over a distance of two bonds, not only between bonds that are separated by one bond, but also the feedback of a bond on itself through neighboring bonds. The third power gives the third order coupling effects and so on. So, truncating the series after a specified number of terms means neglecting all higher order coupling effects. In molecules with bond angles closer to 90° than to 0° or 180° , like proteins, the sum can be truncated after four terms. Little storage is needed, because only the nonzero elements of $\mathbf{A}_n$ have to be stored. $\mathbf{B}_n$ can be stored in a $3K$ array and some extra arrays are needed for temporary storage. Pseudo code for the algorithm is given in the Appendix. The optimized FOR-

TRAN code is implemented as a subroutine in the GROMACS software package.^{12,13}

Parallelization

The inversion through a series expansion also makes parallelization easy. In one timestep, the bonds only influence each other when they are separated by fewer bonds than the highest order in the expansion; with the correction for the rotation this number is doubled. Because of this local coupling a decomposition method can be applied. This can be illustrated by a simple example. Consider a linear bond-constrained molecule of 100 atoms to be simulated on a two-processor computer, using rotation correction and an expansion to the second order of the matrix $(\mathbf{I} - \mathbf{A}_n)^{-1}$. Because the order of the expansion is two, bonds influence each other over a distance of, at most, $2 \cdot (2 + 1) = 6$. The forces can be calculated as in the unconstrained case, but the update of the positions and call of the LINC algorithm must be done for atom 1 to 56 and 44 to 100 on processors 1 and 2, respectively. After the update only the positions of atoms 1 to 50 should be passed from processor 1 to processor 2, as the new positions of atoms 51 to 56 on processor 1 are not accurate. The same holds for atoms 44 to 49 on processor 2. The result of this parallel procedure is identical to the single-processor case. For branched molecules like proteins the same procedure can be used.

Position Langevin Dynamics

Consider a system of N overdamped particles. The equations of motion are given by a position Langevin equation:

$$\frac{d\mathbf{r}}{dt} = \mathbf{\Gamma}^{-1} \mathbf{f} + \sqrt{2\mathbf{\Gamma}^{-1} k_b T} \boldsymbol{\eta} \quad (23)$$

where $\mathbf{\Gamma}$ is the friction matrix and $\boldsymbol{\eta}$ a $3N$ noise vector with no time correlation, Gaussian distributed with $\mu = 0$, and $\sigma = 1$. The algorithm can be derived analogously to the second order case. The result is:

$$\begin{aligned}\mathbf{r}_{n+1} &= (\mathbf{I} - \mathbf{T}_n \mathbf{B}_n) \left(\Delta t \mathbf{\Gamma}^{-1} \mathbf{f}_n + \sqrt{2 \Delta t \mathbf{\Gamma}^{-1} k_b T} \boldsymbol{\eta} \right) \\&\quad + \mathbf{T}_n \mathbf{d}\end{aligned}\quad (24)$$

where $T_n = \Gamma^{-1} B_n^T (B_n \Gamma^{-1} B_n^T)^{-1}$. An attractive property of the projection matrix $P = I - T_n B_n$ is that $P^2 = P$, thus the square root of the projection matrix is the projection matrix itself. This relation is used in eq. (24) because the noise must be multiplied by the square root of the operator that works on the force. When the friction matrix, Γ , is diagonal, the implementation for leap-frog MD can be used, with the masses replaced by friction coefficients. The projection algorithm can also be applied to Monte Carlo simulations, because the distribution of the unconstrained coordinates is not affected by the projection.

Results

A large test system was used to test how the accuracy of the constraints depends on the order of expansion (22) and to compare the speed of the LINCS algorithm with the SHAKE algorithm. Results are given for 1-ps MD runs on lysozyme (129 residue, 1345 bond constraints) with the GRO-MACS package. The protein was solvated in a box of 4535 water molecules and 8 Cl⁻ ions, with Lennard-Jones cut-off of 1 nm and a twin-range Coulomb cut-off (1 and 1.8 nm). The water was constrained with the SETTLE algorithm. After energy minimization and 2-ps equilibration, test runs of 500 steps of 2 fs were performed. The test runs took 1300 seconds on one processor of an R8000 PowerChallenge. Table I and Figure 2 show the results for LINCS and SHAKE. Both routines use

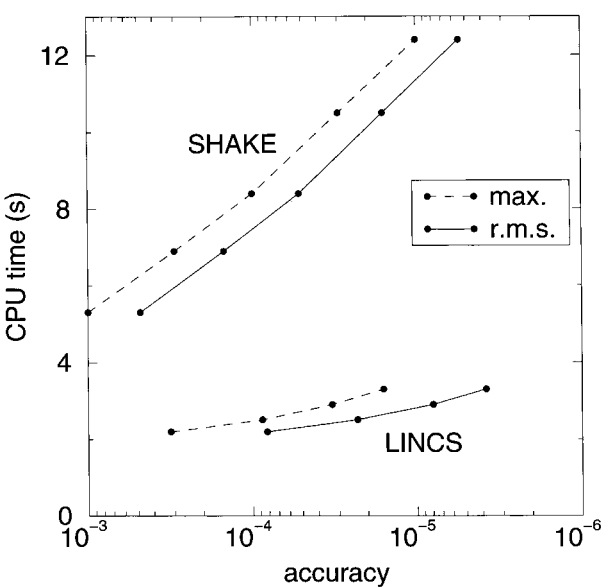


FIGURE 2. The CPU time for the constraint algorithms in lysozyme plotted as a function of the accuracy; that is, the relative deviation of the constraints averaged over 500 steps. The dashed lines show the maximum deviation and the solid lines show the rms deviation for SHAKE and for LINCS.

optimized FORTRAN code. Notice that the difference between the maximum deviation and the rms deviation is a factor 2 for SHAKE and a factor 4 for LINCS. Some runs were also performed without water, giving almost the same accuracy and CPU time for the constraint algorithms as with water

TABLE I.
Relative Deviation of Constraints in Lysozyme (Solvated in 4535 Water Molecules and 8 Cl⁻ Ions)
Averaged Over 500 Steps.^a

Algorithm	Order	Max. dev.	rms. dev.	Time (s)
LINCS (no rotation correction)	2	0.004 9	0.000 59	1.8
	4	0.004 8	0.000 49	2.0
	8	0.004 9	0.000 47	2.4
LINCS	2	0.000 32	0.000 082	2.2
	4	0.000 088	0.000 023	2.5
	8	0.000 016	0.000 003 8	3.3
SHAKE		0.001 0	0.000 48	5.3
		0.000 10	0.000 052	8.4
		0.000 010	0.000 005 5	12.4

^a Maximum and the rms deviation shown for SHAKE and for LINCS both without and with rotation correction. The water was constrained with SETTLE. The second column shows the order after which expansion (22) is truncated. The last column shows the total CPU time for the constraint algorithms. The total run time is 1300 seconds. Vacuum runs give almost identical numbers for the accuracy and CPU time of the constraint algorithms. The total run time of the vacuum system is about 90 seconds.

TABLE II.
Accuracy of Constraints and Total Energy in a 32-Residue $\beta\alpha\beta$ Peptide Using LINCS and SHAKE.^a

Algorithm	Order	Constraints (rms dev.)	E_{total}		E_{kin} rmsf (kJ / mol)
			Drift (kJ / mols)	rmsf (kJ / mol)	
LINCS	2	0.000 088	− 10.26	1.21	39
	4	0.000 027	− 1.67	1.24	41
	8	0.000 004 9	0.25	1.23	41
SHAKE		0.000 089	− 6.81	1.23	39
		0.000 027	− 1.80	1.24	41
		0.000 004 9	0.14	1.27	41

^a All entries are averages over eight vacuum runs in single precision of 4 ps (2000 steps of 2 fs) with different starting structures, without temperature and pressure coupling and without cut-off. The second column shows the order after which expansion (22) is truncated. The third column shows the rms of the relative deviation of the constraints. The fourth column shows the slope of a straight line fitted to the total energy. The last columns show the rms fluctuations in the total and the kinetic energy.

(data not shown), but with a total run time of 90 seconds.

A small test system was used to test the accuracy of the simulation as a function of the accuracy of the constraints. A 32-residue $\beta\alpha\beta$ peptide was chosen, which could be simulated *in vacuo* without cut-off. Simulations in single precision of 4 ps were performed without temperature and pressure coupling to check the conservation of total energy. The tolerance for SHAKE was chosen such that the rms of the relative deviation of the constraints was equal to the LINCS case. Table II shows the results for different accuracies of the constraints for LINCS and SHAKE; each entry is averaged over eight runs of 4 ps. The accuracy of the constraints only influences the drift, not the fluctuations around the drift. Except for the second-order expansion there is little difference between LINCS and SHAKE.

Discussion

LINCS and SHAKE both solve the same nonlinear problem: resetting coupled constraints after an unconstrained update. Just like SHAKE, LINCS is time reversible, because convergence can be obtained by including more matrices in the expansion and using multiple rotation corrections. In single precision one rotation correction is enough to get structures that only differ in machine precision with those obtained by SHAKE. The results show that the LINCS algorithm is three to four times faster than the SHAKE algorithm. In addition,

LINCS has broader convergence conditions than SHAKE, and it can be easily parallelized. The latter is of major importance for simulations of large molecules on parallel computers. For efficient parallelization each molecule must be distributed over several processors, which prevents the straightforward use of the iterative SHAKE routine. Not only will the LINCS algorithm itself decrease the CPU time, but better load balancing of other routines can be achieved as well. In the case of a protein in water on one processor the CPU time for the constraint algorithm is negligible in comparison with the total run time, but LINCS still has the advantage that it has better convergence properties than SHAKE. *In vacuo*, the decrease in total CPU time can be up to 10%.

The algorithm has also been tested on peptides and smaller molecules such as benzene, giving the same three- to four-fold decrease in CPU time. With position Langevin dynamics, SHAKE will not converge for large time steps. When LINCS is used the timestep is no longer limited by the constraint algorithm but rather by physical conditions.

Appendix

VIRIAL AND FREE ENERGY CALCULATION

The virial formulas are given for the case without periodic boundary conditions, for a more detailed description see Allen and Tildesley.¹⁴ The

3×3 virial tensor, Ξ , is defined as:

$$\Xi = -\frac{1}{2} \sum_i^N \mathbf{r}^i \otimes \mathbf{f}_{tot}^i \quad (25)$$

where $\mathbf{f}_{tot}^i$ is the total force working on atom i and $\mathbf{r}^i$ is the position of atom i , $\otimes$ denotes a direct product. For a constrained system at time t_n the virial is:

$$\Xi_n = -\frac{1}{2} \sum_i^N \mathbf{r}_n^i \otimes \left(\mathbf{f}_n^i + \frac{m_i \Delta \mathbf{r}_n^i}{\Delta t^2} \right) \quad (26)$$

where $\mathbf{f}_n^i$ is the unconstrained force working on atom i at time t_n . The $\Delta \mathbf{r}_n$ term in the expression for the constraint force is given by:

$$\Delta \mathbf{r}_n = \mathbf{r}_{n+1} - \left(\mathbf{r}_n + \Delta t \mathbf{v}_{n-\frac{1}{2}} + (\Delta t)^2 \mathbf{M}^{-1} \mathbf{f}_n \right) \quad (27)$$

One way to calculate the free energy difference between two systems or two states of a system is

to make the Hamiltonian an analytical function of a coupling parameter α , where $\alpha = 0$ corresponds to system A and $\alpha = 1$ to system B.¹⁵ The constraint distances may be a function of the coupling parameter:

$$\mathbf{d}(\alpha) = (1 - \alpha) \mathbf{d}_A + \alpha \mathbf{d}_B \quad (28)$$

where $\mathbf{d}_A$ and $\mathbf{d}_B$ are the lengths of the constraints in system A and B, respectively. For the contribution of the constraints to the free energy, F^{con} , the derivative with respect to α of the added term to the potential in eq. (3) is needed:

$$\begin{aligned} \frac{dF^{con}}{d\alpha} &= -\frac{\partial(\boldsymbol{\lambda} \cdot \mathbf{g})}{\partial \alpha} = -\boldsymbol{\lambda} \cdot \frac{\partial \mathbf{g}}{\partial \alpha} \\ &= -\boldsymbol{\lambda} \cdot (\mathbf{d}_B - \mathbf{d}_A) \end{aligned} \quad (29)$$

The discretized Lagrange multipliers are already calculated by the LINCS algorithm.

PSEUDO CODE

```

LINCS(x, xp, invmass, K, nrec, atom1, atom2, length, ncc, cmax, con, Sdiag, coef)
# x[N, 3]          old positions of the atoms, N is the number of atoms
# xp[N, 3]         input: new unconstrained positions,
#                  output: the constrained positions
# invmass[N]       inverse masses of the atoms
# K                number of constraints
# nrec             order after which the power series is truncated
# atom1[K]         first atom of the constraint
# atom2[K]         second atom of the constraint
# length[K]        length of the bond
# ncc[K]           number of constraints connected to a constraint
# cmax             maximum number of constraints coupled to one constraint
# con[K, cmax]     index of the constraints connected to a constraint
# Sdiag[K]         1 / sqrt(invmass[atom1] + invmass[atom2])
# coef[K, cmax]    coef[i, j] = sign * invmass[c] * Sdiag[i] * Sdiag[con[i, j]],
#                  c is the atom coupling constraints i and con[i, j],
#                  sign = -1 if atom1[i] = atom1[con[i, j]] or atom2[i] = atom2[con[i, j]]
#                  else sign = 1
{ real B[K, 3]      # directions of the constraints
  real A[K, cmax]   # normalized constraint coupling matrix
  real rhs[2, K]    # right hand side of the matrix equation,
                  # two arrays needed to iterate
  real sol[K]       # solution array of the matrix equation
  int i, j, k, n, a1, a2
  real len, p

  for i = 1 to K
  { for j = 1 to 3
    { B[i, j] = x[atom1[i], j] - x[atom2[i], j] }
  }

```



```

len = sqrt(sqr(B[i, 1]) + sqr(B[i, 2]) + sqr(B[i, 3]))
for j = 1 to 3
  { B[i, j] = B[i, j] / len }
}
for i = 1 to K
  { for n = 1 to ncc[i]
    { k = con[i, n]
      A[i, n] = coef[i, n] * (B[i, 1] * B[k, 1] + B[i, 2] * B[k, 2] + B[i, 3] * B[k, 3]) }
    a1 = atom1[i]
    a2 = atom2[i]
    rhs[1, i] = Sdiag[i] * (B[i, 1] * (xp[a1, 1] - xp[a2, 1]) +
                          B[i, 2] * (xp[a1, 2] - xp[a2, 2]) +
                          B[i, 3] * (xp[a1, 3] - xp[a2, 3]) - length[i])
    sol[i] = rhs[1, i]
  }
}
call SOLVE(xp, invmass, K, nrec, atom1, atom2, ncc, con, Sdiag, B, rhs, sol)

```

CORRECTION FOR ROTATIONAL LENGTHENING

```

for i = 1 to K
  { a1 = atom1[i]
    a2 = atom2[i]
    p = sqrt(2 * sqr(length[i]) - sqr(xp[a1, 1] - xp[a2, 1]) -
            sqr(xp[a1, 2] - xp[a2, 2]) -
            sqr(xp[a1, 3] - xp[a2, 3]))
    rhs[1, i] = Sdiag[i] * (length[i] - p)
    sol[i] = rhs[1, i]
  }
  call SOLVE(xp, invmass, K, nrec, atom1, atom2, ncc, con, Sdiag, B, rhs, sol)
}

```

SOLVE(xp, invmass, K, nrec, atom1, atom2, ncc, con, Sdiag, B, rhs, sol)

```

{ int i, j, n, rec, w, a1, a2

w = 2
for rec = 1 to nrec
  { for i = 1 to K
    { rhs[w, i] = 0
      for n = 1 to ncc[i]
        { rhs[w, i] = rhs[w, i] + A[i, con[i, n]] * rhs[3 - w, con[i, n]] }
        sol[i] = sol[i] + rhs[w, i]
      }
    w = 3 - w
  }
  for i = 1 to K
    { a1 = atom1[i]
      a2 = atom2[i]
      for j = 1 to 3
        { xp[a1, j] = xp[a1, j] - invmass[a1] * B[i, j] * Sdiag[i] * sol[i]
          xp[a2, j] = xp[a2, j] + invmass[a2] * B[i, j] * Sdiag[i] * sol[i] }
        }
    }
}

```

References

1. J. P. Ryckaert, G. Ciccotti, and H. J. C. Berendsen, *J. Comput. Phys.*, **23**, 327 (1977).
2. S. Miyamoto and P. A. Kollman, *J. Comput. Chem.*, **13**, 952 (1992).
3. E. Barth, K. Kuczera, B. Leimkuhler, and R. D. Skeel, *J. Comput. Chem.*, **16**, 1192 (1996).
4. B. J. Leimkuhler and R. D. Skeel, *J. Comput. Phys.*, **112**, 117 (1994).
5. S. E. DeBolt and P. A. Kollman, *J. Comput. Chem.*, **14**, 312 (1993).
6. R. Edberg, D. J. Evans, and G. P. Morriss, *J. Chem. Phys.*, **84**, 6933 (1986).
7. A. Baranyai and D. J. Evans, *Mol. Phys.*, **70**, 53 (1990).
8. M. Yoneya, H. J. C. Berendsen, and K. Hirasawa, *Mol. Simul.*, **13**, 395 (1994).
9. J. T. Slusher and P. T. Cummings, *Mol. Simul.*, **18**, 213 (1996).
10. H. Bekker, *Molecular Dynamics Simulation Methods Revised*, Ph.D. thesis, University of Groningen, Groningen, The Netherlands, 1996.
11. L. Landau and E. Lifshitz, *Mechanics*, Pergamon Press, Oxford, 1961.
12. H. J. C. Berendsen, D. van der Spoel, and R. van Drunen, *Comput. Phys. Commun.*, **19**, 43 (1995).
13. D. van der Spoel, H. J. C. Berendsen, A. R. van Buuren, E. Apol, P. J. Meulenhoff, A. L. T. M. Sijbers, and R. van Drunen, *GROMACS User Manual*, Nijenborgh 4, 9747 AG Groningen, The Netherlands, 1995. Internet: <http://rugmd0.chem.rug.nl/~gmx>.
14. M. P. Allen and D. J. Tildesley, *Computer Simulations of Liquids*, Oxford Science Publications, Oxford, 1987.
15. W. F. van Gunsteren, In *Computer Simulation of Biomolecular Systems, Theoretical and Experimental Applications: Successes and Problems*, W. F. van Gunsteren and P. Weiner, Eds., Escom, Leiden, 1989, p. 27.